

GEP OOP interview Q&A

Ojas Srivastava · Only questions from real GEP / on-campus reports

Sarthak Chauhan (SVNIT → PPO) · GFG intern/SDE-1 · TalentBattle Airoli

How GEP runs OOP (10 min is common).

They do not want textbook lists. They pick **one pillar**, push for a **real-world example**, then jump to **polymorphism + inheritance**. Round 2 adds **design a bank / Zoomcar** and keyword drills (static, friend, access specifiers).

Rule: every answer gets a LogiFlow or IFFCO example. Speak in 2–4 sentences, then stop.

1. The four pillars — with real-world examples

Asked at GEP: Sarthak Chauhan, R1 — “Explain pillars of OOP” + heavy follow-up on polymorphism and inheritance.

Q. Explain the four pillars of OOP with real-world examples.

Typical: Lists four words and stops.

You say. Encapsulation — my route service hides Redis; callers only see `getRoutes()`. Abstraction — a steering wheel: you turn it without knowing the rack. Inheritance — `Vehicle` → `Car/Truck` share start/stop, each adds its own behaviour. Polymorphism — I call `cost()` and a railway route and a road route compute it differently.

Q. Go deeper on polymorphism and inheritance.

Typical: Repeats definitions louder.

You say. Inheritance is “is-a” reuse — three LogiFlow pipelines share the same output shape. Polymorphism is one interface, many behaviours — compile-time is overloading, runtime is overriding through a base type. The ranker calls the same method on railway, comparator and hybrid output; it only cares about time, cost and delay.

Q. What is the purpose of OOP?

Asked at GEP: GFG intern, R2 (10 min round).

Typical: “To make code reusable.”

You say. To model a system as objects that own data and behaviour. Change stays local — I can swap the railway pipeline without touching the ranker. Reuse is a side effect; the goal is a system you can reason about and change safely.

Q. Abstraction vs abstract class vs encapsulation.

Asked at GEP: GFG SDE-1, R2 — OOP-heavy round.

Typical: Uses abstraction and encapsulation interchangeably.

You say. Abstraction is the design idea — show what, hide how. An abstract class is the language tool — cannot be instantiated, can hold unimplemented methods. Encapsulation bundles data with methods and restricts direct access. Abstraction hides complexity; encapsulation hides data.

Q. Access specifiers — public, private, protected.

Asked at GEP: TalentBattle ASE, Airoli 2023.

Typical: Names three words, no scope.

You say. Public — visible everywhere. Private — only inside the class. Protected — class and subclasses. In Java, default is package-private. I keep fields private and expose behaviour through methods, not raw state.

2. Keywords they drill

Q. What does `static` mean?

Asked at GEP: GFG SDE-1, R2.

Typical: “It means constant.”

You say. Static belongs to the class, not any object — one copy shared by all instances. A static method runs without creating an object, so it cannot touch instance fields. I use it for utility helpers and shared counters.

Q. What is a friend function? (C++)

Asked at GEP: GFG SDE-1, R2.

Typical: Silence or confuses with inheritance.

You say. A function declared inside a class but not a member — allowed to access private and protected data. Used for operator overloading across two classes. It deliberately breaks encapsulation, so it should be rare. C++ has it; Java does not.

Q. Why is `main` public? Why `String[] args`?

Asked at GEP: GFG Associate SD, R2.

Typical: “Because Java requires it.”

You say. Public because the JVM calls it from outside the class. Static so it runs without creating an object. `String[] args` holds command-line arguments — an array because there can be many.

Q. Difference between class and object.

Typical: “Class is blueprint, object is instance” with no example.

You say. A class is the template; an object is one live instance with its own field values. `Route` is the class; one corridor from Mumbai to Delhi with cost 1200 is an object. Many objects, one class definition.

Q. Method overloading vs overriding.

Typical: Mixes them up.

You say. Overloading — same method name, different parameters, resolved at compile time. Overriding — subclass replaces a parent method, resolved at runtime through the actual object type. Overloading is convenience; overriding is polymorphism.

Q. Interface vs abstract class.

Typical: “They are the same.”

You say. An interface is a pure contract — methods without implementation (pre-Java 8). An abstract class can have both abstract and concrete methods plus fields. I use an interface when unrelated classes must share behaviour; abstract class when they share code too.

Q. Composition vs inheritance.

Typical: “Always inherit.”

You say. Inheritance is “is-a”; composition is “has-a”. A `Ranker` *has* pipeline objects rather than inheriting from them — easier to swap railway for hybrid without a deep hierarchy. Prefer composition when behaviour varies at runtime.

3. OOP design questions — whiteboard style

Asked at GEP: GFG SDE-1, R2: “Design a bank using OOP.” GFG SDE-1, R1: “Design Zoomcar.”

Q. Design a bank account system using OOP.

Typical: One giant `Bank` class with everything inside.

You say. Abstract Account with private balance, deposit() and withdraw(). Savings and Current extend it and override withdraw() for different overdraft rules — runtime polymorphism. Balance changes only through methods so I can validate, log and reject. A Transaction class records each operation for audit.

Q. Design Zoomcar / a car rental system.

Typical: Jumps to SQL tables with no classes.

You say. Entities: User, Car, Location, Booking, Payment. A Booking links user, car and time range; the constraint is no overlapping bookings for the same car. Search by location and window — index on car ID plus start time. For double-booking I use a transaction lock or a unique constraint on the slot.

Q. Design a timetable using OOP + database.

Asked at GEP: GFG Associate SD — timetable DB design.

Typical: One table with teacher, subject, room, batch as columns repeated everywhere.

You say. Classes: Teacher, Subject, Room, Batch, Slot (day + period). TimetableEntry references all four plus slot. Uniqueness rules enforce one teacher per slot, one room per slot. That is encapsulation of scheduling rules in the schema and service layer.

Q. Two users grab the last item in a cart at the same time — what happens?

Asked at GEP: GFG SDE-1, R2 — race condition.

Typical: “The second user gets an error.”

You say. Race condition — both read stock as 1, both write 0. Fix: atomic check-and-decrement — a transaction with row lock, or UPDATE ... SET stock = stock - 1 WHERE stock > 0 and check rows affected. Loser gets “out of stock.” Same pattern as LogiFlow: SQL is source of truth, Redis cache never decides inventory.

4. OOP tied to your projects

Q. How did you use OOP in LogiFlow?

Typical: “We used FastAPI so it was object-oriented.”

You say. Separate pipeline objects produce corridor candidates; a ranker object scores them on time, cost and delay. Each pipeline encapsulates its own data fetch and transform — the ranker only sees a common result interface. That let us add hybrid without rewriting railway.

Q. How did you do login and authentication? (project follow-up pattern)

Asked at GEP: Sarthak Chauhan, R1 — deep dive on login/auth in his project.

Typical: “Firebase login button.”

You say. On Community Hero: Firebase Auth issues a token on the client; Express verifies it on every protected route before Firestore. Roles are checked server-side — a citizen cannot hit admin endpoints. Encapsulation: auth logic lives in middleware, routes never trust the client.

Q. Give a real-world OOP example from your internship.

Typical: Vague about “building APIs.”

You say. At IFFCO I built Express route handlers as separate modules — each endpoint encapsulated validation, SQL and error handling. Shared middleware handled auth and logging so individual routes stayed small. That is separation of concerns through object/module boundaries.

5. Language comparison (OOP context)

Q. Python vs Java — when do you pick each?

Asked at GEP: TalentBattle ASE, Airoli 2023.

Typical: “Python is easier.”

You say. Python is dynamically typed and fast to write — my FastAPI backend. Java is statically typed, bytecode-compiled, catches type errors at compile time and runs faster on large systems. Python for data pipelines and APIs; Java when the team needs strict contracts at scale.

Q. MySQL vs MongoDB — and have you used Mongo?

Asked at GEP: TalentBattle ASE, Airoli 2023.

Typical: Bluffs Mongo experience.

You say. MySQL is relational — fixed schema, joins, ACID. MongoDB is document-based — flexible schema, horizontal scale, no joins. I have not used Mongo; closest is Firestore on Community Hero. Relational work is MySQL at IFFCO and SQL in LogiFlow.

6. Quick reference — say this if stuck

Encapsulation	Hide internal state; expose behaviour through methods.
Abstraction	Show essential operations; hide implementation detail.
Inheritance	Reuse and extend — “is-a” relationship.
Polymorphism	One interface, many implementations — overloading vs over-riding.
Static	Belongs to class, not instance; no instance fields.
Private fields	Always; validate in setters and business methods.

7. Last check before OOP round

- Lead with a **LogiFlow example** within the first two sentences.
- If they say “go deeper,” give **one more sentence**, not a lecture.
- Design questions: name **classes first**, then one constraint, then one edge case.
- Never say static means constant. Never mix abstraction and encapsulation.
- If you do not know (friend in Java): say what language has it and move on.